Papillon

SAE 3.1 – FI

Logiciel de gestion de rappels flottants et persistants

Valentin WIECZOREK

Benicia BOUNNI LOUBELO

Camille LEVCHEVALIER

Année universitaire : 2025 – 2026

Enseignant référent : Luc HERNANDEZ
Date de rendu : Dimanche 26 octobre 2025,

TABLE DES MATIÈRES

---

1. Introduction

1.1 Contexte du projet
1.2 Objectifs généraux

2. Présentation du logiciel

2.1 Description des fonctionnalités principales
2.2 Exemple d'utilisation

3. Conception du programme

3.1 Structure générale
3.2 Diagramme de classes simplifié
3.3 Gestion de la base de données
3.4 Diagramme complet de la base et des relations

4. Développement et mise en œuvre

4.1 Gestion des erreurs et messages à l'utilisateur
4.2 Compilation et exécution via Makefile et JAR

5. Design et Ergonomie

5.1 Analyse des choix d'interface
5.2 Améliorations envisagées

6. Conclusion

6.1 Bilan de Camille
6.2 Bilan de Benicia
6.3 Bilan de Valentin

1. Introduction

1.1 Contexte du projet

Ce projet nous a été demandé dans le cadre d'une situation d'apprentissage et d'évaluation (SAE) du semestre 3 pour la promotion BUT 2 informatique de l'année 2025-2026. Ce projet consiste en la création d'une application en java complet afin de mettre en pratique la grande majorité des notions vues jusqu'à aujourd'hui. Le sujet, intitulé Papillon consiste e la création d'une application toujours visible au premier plan dans laquelle l'utilisateur peut inscrire une liste de rappels qu'il doit accomplir. Il doit pouvoir ajouter, supprimer et modifier des rappels comme il le souhaite comme une « to do list ».

1.2 Objectifs généraux

Les objectifs et fonctionnalités principales attendues sont :

La création, la consultation, la modification et la suppression de rappels (opérations CRUD)

L'affichage d'une liste compacte et ordonnée des rappels selon un rang défini (par urgence ou chronologie)

La gestion de thèmes visuels distinctifs (au moins cinq) permettant de personnaliser les rappels

La persistance des données dans une base de données relationnelle

Une interface graphique intuitive, reposant sur les bibliothèques standards de Java (Swing ou AWT).

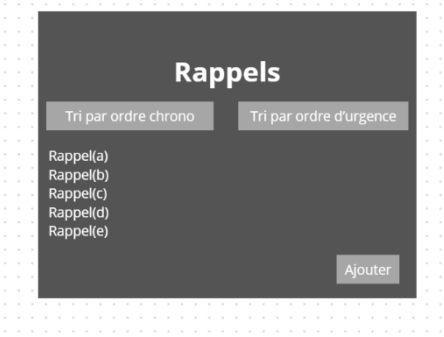
Un affichage persistant au premier plan impliquant la nécessité de minimiser l'espace occupé sur l'écran

1.3 Wireframe

Voici une **représentation visuelle simplifiée** de l'interface imaginé en début de SAE.
C'est une **ébauche** qui montre **l'organisation des éléments à l'écran**. Ici on ne se soucis pas des couleurs, de la police etc…

Nous étions d'abord parti sur cette première conception visuelle de l'interface:

Affichage des rappels

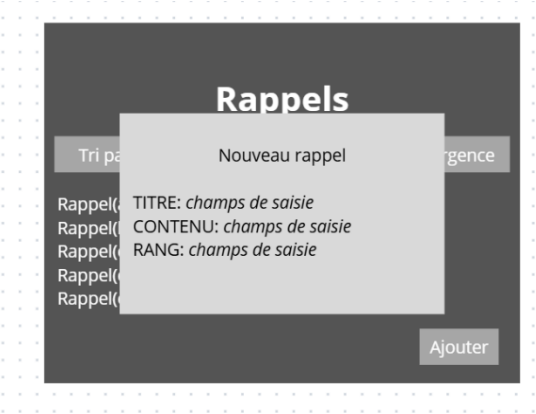


Le premier écran représente l'écran affichant tous les rappels. On a évité un affichage dense afin de ne pas encombrer l'affiche pour l'utilisateur. A la place on partait sur le principe de pouvoir consulter un rappel en détail lorsque l'on clique dessus.

Affichage d'un rappel en particulier:

Le deuxième montre le détail d'un rappel précis.

Création d'un rappel



Et le troisième montre l'affichage dans le cas où un nouvel rappel souhaite être créé.

2. Présentation du logiciel

2.1 Description et fonctionnalités principales

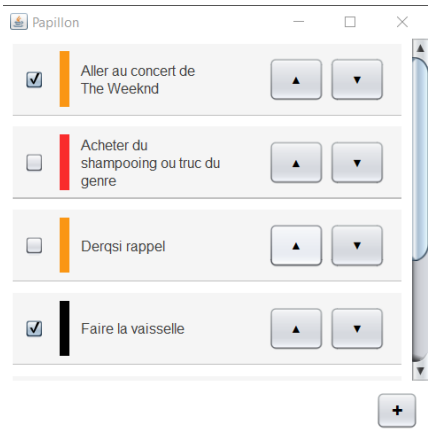
Lorsqu'un utilisateur ouvre le logiciel Papillon, il accède directement à l'écran principal où tous les rappels existants sont affichés de manière compacte et lisible, toujours visibles au premier plan. Un petit bouton portant un symbole « + » permet d'ajouter rapidement un nouveau rappel ; en cliquant dessus, l'utilisateur est redirigé vers une page dédiée à la création de rappel.

Chaque rappel doit respecter certaines contraintes imposées par la base de données : par exemple, le titre d'un rappel ne peut pas être nul, et si l'utilisateur laisse ce champ vide, une boîte de dialogue apparaît pour l'avertir de l'erreur avec un bouton « OK ». De même, le champ « contenu » est limité à 200 caractères maximum, garantissant que les rappels restent concis et lisibles. Une fois qu'un rappel est correctement saisi et validé, les champs du formulaire se vident automatiquement, permettant à l'utilisateur d'ajouter un autre rappel sans revenir en arrière. Lors de la création, il est également possible de choisir un thème pour le rappel, afin de le distinguer visuellement selon l'importance ou le type de tâche. Sur l'écran principal, l'utilisateur peut réorganiser ses rappels en déplaçant un rappel au-dessus d'un autre pour marquer son importance relative.

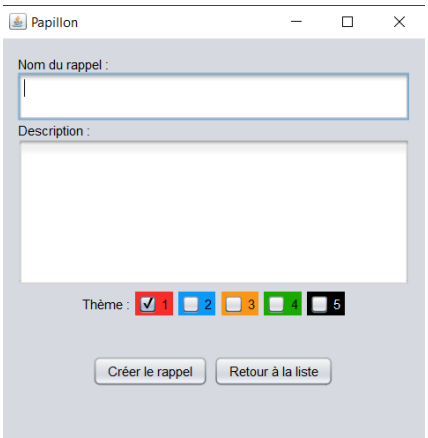
Chaque rappel dispose d'une case à cocher pour indiquer qu'il a été effectué, sans pour autant le supprimer de la liste, afin de conserver un historique. En cliquant sur un rappel, celui-ci s'ouvre dans un affichage agrandi, présentant son contenu complet ainsi que deux boutons : « Supprimer » et « Modifier ». Si l'utilisateur choisit de supprimer un rappel, une boîte de dialogue lui demande confirmation avant toute action définitive. S'il souhaite modifier un rappel, il est renvoyé vers l'interface de création, mais cette fois-ci avec les champs préremplis correspondant au rappel sélectionné, permettant une modification rapide et intuitive.

2.2 Exemple d'utilisation

Un utilisateur souhaite ajouter un rappel pour ne pas oublier de rendre un devoir important. En ouvrant le logiciel, il arrive directement sur l'écran principal où s'affichent déjà plusieurs rappels qu'il avait créés auparavant.

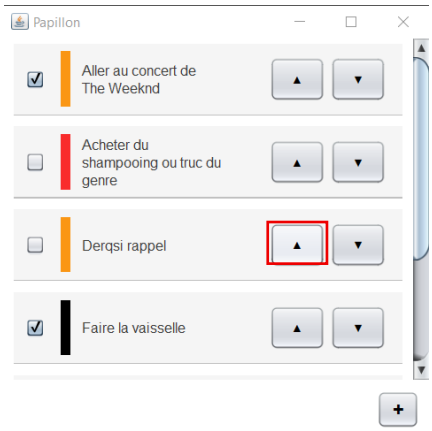


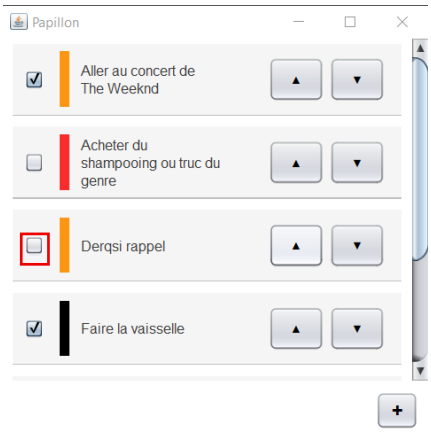
Il clique sur le petit bouton « **+** » situé en bas à droite de l'écran pour créer un nouveau rappel. Une nouvelle page s'ouvre (en réalité il s'agit de la même fenêtre) il saisit le titre **« Rendre le devoir de bases de données »** et écrit dans le champ contenu : **« Déposer le devoir avant vendredi 18h »**.
Il choisit ensuite le thème bleu par exemple qui signifie pour lui études pour le classer parmi ses rappels universitaires.
En validant, le rappel est enregistré et les champs du formulaire se vident automatiquement, lui permettant d'ajouter un autre rappel s'il le souhaite.



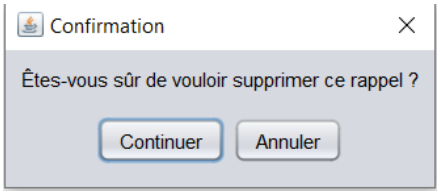
De retour sur l'écran principal, le nouveau rappel apparaît immédiatement dans la liste. Il décide de le faire glisser en haut pour le mettre en priorité.

Après avoir réalisé sa tache il coche la case se trouvant a gauche du rappel pour indiquer que la tache n'est plus à faire. Pour autant, cette manipulation ne supprime pas le rappel car il se pourrait qu'il en ait encore besoins.





Plusieurs jours plus tard, l'utilisateur ne se souviens plus a quoi correspond le rappel « rendre le devoir en base de données » et click donc déçu pour en apprendre plus sur le contenu. Là, il prend la décision de supprimer le rappel. Une boite de dialogue apparait pour confirmer le choix de l'utilisateur.



3. Conception du programme

3.1 Structure général

Le projet est conçue selon l'architecture MVC (Modèle-Vue-Contrôleur) afin de séparer clairement les données, l'interface graphique et la logique métier, ce qui facilite la maintenance et l'évolution du logiciel.

La partie Modèle comprend les classes de base, dont la classe Rappel représentant un rappel avec ses attributs principaux tels que l'identifiant, le titre, le contenu, le thème, l'importance et le statut, ainsi que la classe ListeRappels qui gère la collection de rappels et la classe DatabaseManager, responsable de la connexion à la base de données MariaDB via JDBC.

La Vue, quant à elle, regroupe les classes d'interface graphique qui permettent l'interaction avec l'utilisateur : MenuPapillon constitue la fenêtre principale et utilise un CardLayout pour naviguer entre les différents panneaux, tandis que JListeRappels affiche la liste des rappels, JRappel représente un rappel individuel sous forme graphique, JFicheRappel montre le détail d'un

rappel, et JCreerRappel et JModifRappel permettent respectivement la création et la modification de rappels. Les messages d'erreur sont gérés par la classe JErreur.

La partie Contrôleur inclut les classes de gestion des données, telles que LectureRappel pour la lecture des rappels depuis la base de données, EcritureRappel pour leur écriture, et ModifierRappel pour les mises à jour.

Des classes utilitaires comme Confirmation et Observateur/ObservateurConfirm prennent en charge la gestion des boîtes de dialogue et des événements utilisateur.

L'interface est toujours affichée au premier plan grâce à setAlwaysOnTop, et la navigation entre les différentes vues est fluide grâce au CardLayout. Chaque vue graphique communique avec la base de données via DatabaseManager et utilise des observateurs pour gérer les événements UI, assurant ainsi une interaction réactive et sécurisée. La base de données est structurée autour d'une table Rappel comportant les champs id (clé primaire), titre, contenu, theme, importance et statut, permettant de stocker de manière persistante toutes les informations nécessaires.

Création de rappels

Pour la création de rappels, il est nécessaire d'utiliser des TextArea de taille suffisamment grande afin de permettre une saisie confortable du texte. Les boutons doivent être bien visibles, idéalement positionnés en bas de la fenêtre.
Chaque thème est associé à une couleur spécifique afin de rendre l'interface plus claire et plus lisible.
La fenêtre reste fonctionnelle même lorsqu'elle est réduite, bien que cette configuration puisse parfois être peu adaptée selon le contenu.
Enfin, l'utilisateur doit pouvoir choisir de ne pas revenir à la liste principale après la création d'un rappel, afin de pouvoir enchaîner la création de plusieurs rappels d'affilée.

Modification des rappels

La fenêtre de modification des rappels reprend la même structure que celle de création, à la différence que les informations du rappel sont déjà préremplies.
Une fois la modification validée, l'utilisateur est redirigé vers la fiche du rappel concerné.

Liste des rappels

Dans la liste des rappels, le thème est mis en évidence grâce à un rectangle de couleur.
Les informations les plus importantes sont placées à gauche afin qu'elles restent visibles même lors du redimensionnement de la fenêtre.
Un bouton de coche permet de marquer les rappels réalisés, et des boutons de réorganisation placés à droite permettent d'arranger l'ordre des rappels (fonctionnalité utilisée plus occasionnellement).
Le clic sur l'ensemble du rectangle d'un rappel est rendu intuitif et redirige directement vers la fiche détaillée du rappel.
Enfin, l'utilisation de balises HTML sur les titres permet le retour à la ligne automatique, améliorant ainsi la lisibilité.

Fiche rappel

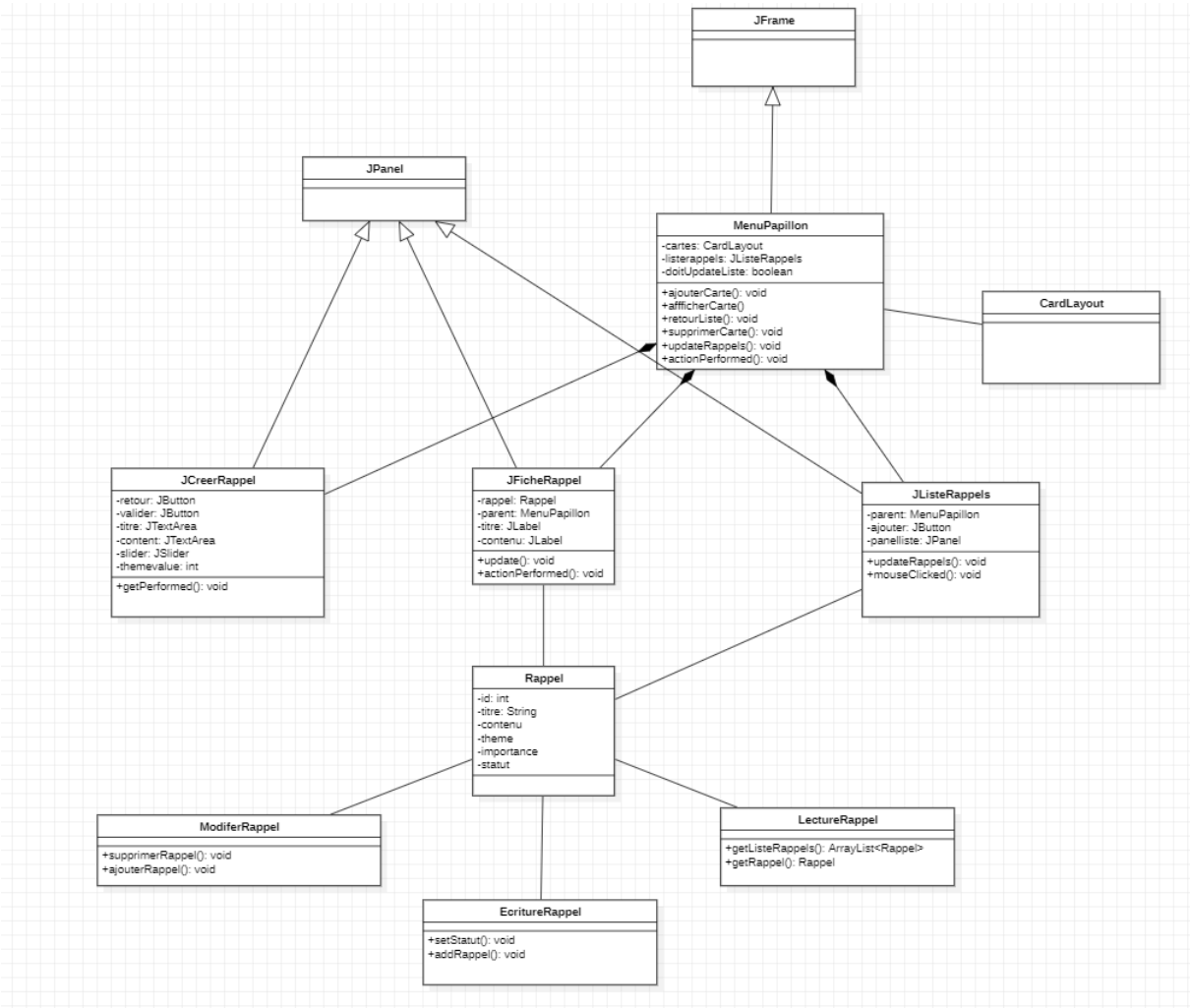
Sur la fiche d'un rappel, le titre est affiché en grand et peut contenir des balises HTML pour permettre le retour à la ligne sur le titre comme sur le contenu.

Un rectangle coloré à gauche rappelle le thème du rappel.
En bas de la fenêtre, trois boutons sont présents : retour à la liste, modifier (qui renvoie à la fenêtre d'édition du rappel) et supprimer.
Le bouton supprimer est mis en évidence en rouge pour souligner son importance.

Sur l'ensemble du projet

Une attention particulière a été portée à la cohérence visuelle entre toutes les fenêtres de l'application, afin de conserver une identité partagée à travers la navigation.
Des bordures et marges internes (Insets) ont été utilisées pour bien séparer les éléments et rendre l'ensemble de l'interface plus lisible et harmonieux.

3.2 Diagramme de classes



3.3 Gestion de la base de données : Schéma de la Base de Donnée Papillon

Le schéma de la base de donnée si dessous n'est pas exactement celui que nous avons choisis au début du projet. En effet, le projet nous imposait de ranger les rappels selon un ordre précis en rang. Nous avions donc au départ géré cet ordre de priorité par un rang que l'utilisateur devait

choisir a la création du rappel : un ordre de priorité qui allait de arbitrarement de 0 à 10. Par la suite on a retiré le champ “priorité” par souci de laisser le choix à l’utilisateur le choix de l’interprétation du tri, de cette manière, il n'est plus nécessaire d’avoir les champs “Date” et “Ordre de priorité”.

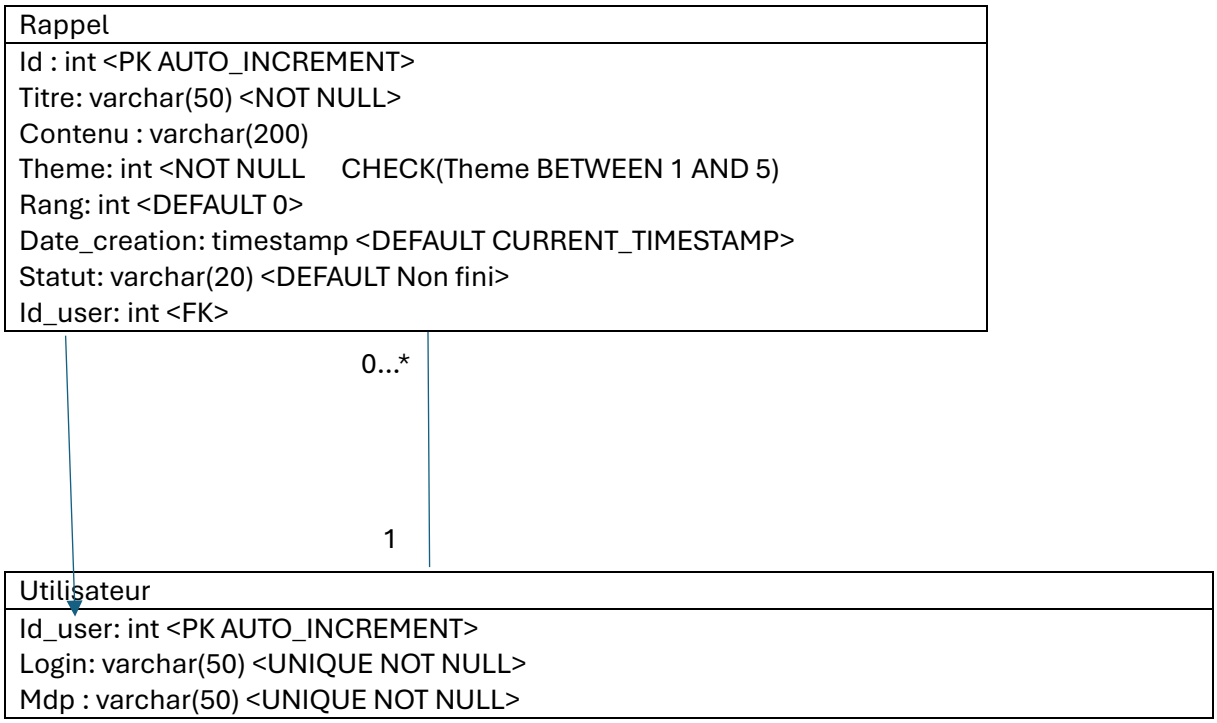
Dans le cas ou l’utilisation de l’application est unique (une seule personne), l’application ne nécessite qu’une table, celle des informations des rappels. Nous verrons ensuite le cas du schéma de la base de donnée pour une utilisation commune (dans le cas où l’application devrait être utilisée par plusieurs personne.

Rappel
Id : int <PK AUTO_INCREMENT> Titre: varchar(50) <NOT NULL> Contenu : varchar(200) Theme: int <NOT NULL CHECK(Theme BETWEEN 1 AND 5) Rang: int <DEFAULT 0> Date_creation: timestamp <DEFAULT CURRENT_TIMESTAMP> Statut: varchar(20) <DEFAULT Non fini>

```
CREATE TABLE Rappel (
   Id INT PRIMARY KEY AUTO_INCREMENT,
   Titre VARCHAR(50) NOT NULL,
   Contenu VARCHAR(200),
   Theme INT NOT NULL,
   Date_creation TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   Statut VARCHAR(20) DEFAULT ‘Non fini’,
   CHECK (Theme BETWEEN 1 AND 5),
);
```

Ici, l’id sert à manipuler un rappel. Le thème est un Integer compris entre 1 et 5 (pourra être ajuster selon le nombre de themes implementés. Et enfin le rang sera determiné d’abord par l’importance du rappel puis de sa date de création.

Pour une utilisation commune, il suffit de rajouter une table utilisateur pour que ce dernier accède à ses rappel.



4. Développement et mise en œuvre

4.1 Gestion des erreurs et messages à l'utilisateur

La gestion des erreurs a été pensée de manière à être à la fois claire pour l'utilisateur et précise pour le développeur. Lorsqu'une exception est détectée dans le programme, celle-ci est interceptée par un bloc try/catch afin d'éviter un arrêt brutal du logiciel. Dans la plupart des cas, lorsqu'une erreur survient, une fenêtre de dialogue s'affiche à l'écran pour prévenir l'utilisateur.

Cette fenêtre, gérée par la classe JErreur, contient un message explicite décrivant le problème de façon vulgarisée et un bouton « OK » permettant de fermer la boîte de dialogue après lecture. L'objectif est d'offrir un retour visuel immédiat à l'utilisateur tout en maintenant une expérience fluide et compréhensible.

Parallèlement, le programmeur dispose d'informations techniques plus détaillées grâce à l'instruction e.printStackTrace();, utilisée dans le bloc catch. Cette commande affiche dans la console la trace complète de l'erreur, ce qui facilite le débogage et la maintenance du code sans perturber l'utilisateur final. Au final, les erreurs sont gérées de telle sorte a ce qu'elles soit claires pour l'utilisateur et qu'il ait un feedback et comprenne que quelque chose ne fonctionne pas tandis que pour le développeur, il n'a qu'a aller dans la console pour en apprendre plus sur l'erreur.

Ce système de dialogue (JErreur) est repris dans « Confirmation.java » qui est appelé une fois que l'utilisateur souhaite supprimer un rappel. Sur le principe, il s'agit du même fonctionnement mais ici l'utilisateur a le choix entre plusieurs boutons : « continuer »et « annuler ». C'est assez

explicite : un observateur attend le click de l'utilisateur sur un des deux boutons et continue l'action « DELETE » dans la base de donnée selon le bouton cliqué.

4.2 Compilation et exécution via Makefile et JAR

Au lieu de taper plusieurs commandes à la main dans le terminal, un makefile a été créé : c'est ainsi qu'on lance le programme. Le makefile a été pensé de telle sorte a ce que l'exécution soit facile pour Windows et linux (bien qu'il faille quand même taper une ligne de commande dans le terminal). En effet, le fichier détecte quel OS l'utilisateur utilise. Ainsi il suffit d'utiliser « make run » pour compiler et lancer. Cette commande verifie si le dossier « build » existe sinon il le créer et y place tous les fichiers compilés puis lance le main.

Pour exécuter le programme, il suffit d'ouvrir un terminal dans le dossier du projet et de taper la commande « make ». Cette commande effectue automatiquement toutes les étapes nécessaires : elle compile les fichiers sources Java, crée le dossier de build si nécessaire, et génère un fichier exécutable Papillon.jar. Une fois le JAR créé, le programme se lance

5. Design et Ergonomie

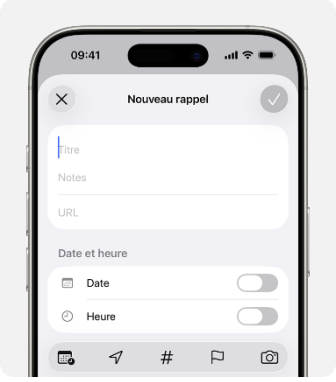
5.1 Analyse des choix d'interface

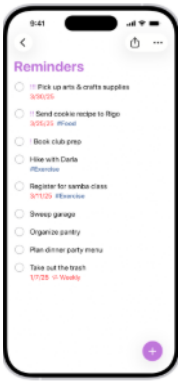
Le design et l'ergonomie de l'application ont été pensés pour offrir une expérience simple, claire et efficace à l'utilisateur. L'application est toujours affichée **au premier plan** ("always on top"), ce qui permet de garder ses rappels visibles à tout moment, même lorsqu'on travaille sur d'autres logiciels. Cela facilite la gestion du temps et des tâches quotidiennes sans avoir à ouvrir ou rechercher la fenêtre du programme.

L'interface est composée de **trois écrans principaux** :

- la **liste des rappels**, qui présente tous les rappels de manière compacte et colorée ;
- la **fenêtre de création de rappel**, où l'utilisateur peut saisir un titre, une description et choisir un thème coloré ;
- et la **fiche de rappel**, qui affiche les détails d'un rappel et permet de le modifier ou de le supprimer.

Chaque **thème** est représenté par une **couleur vive**, ce qui permet d'identifier rapidement la catégorie d'un rappel. Les boutons sont larges, bien espacés, et les actions principales (ajouter, modifier, supprimer) sont facilement accessibles. Les flèches permettent de changer la priorité ou l'ordre des rappels, tandis qu'un simple clic sur le bouton « + » ouvre la fenêtre de création. Nous nous sommes inspiré de l'application mobile "Rappels" de Iphone, dont le placement du bouton permettant d'ajouter qui s'est transformé en bouton "+" et a été placé en bas à droite de l'écran.





L'ensemble du design mise sur la **simplicité et la rapidité d'usage** : pas de menus complexes ni de sous-niveaux de navigation, tout est accessible en un clic. L'objectif est que l'utilisateur puisse comprendre l'utilisation au premier usage.

5.2 Améliorations envisagées

Plusieurs **améliorations** sont envisagées pour faire évoluer l'application**.**

Tout d'abord, une meilleure gestion du temps pourrait être ajoutée : la date de création est enregistré dans la base de donnée. Un système de tri par date pourrait être implémenté dans le but de trier différemment les rappels. Actuellement, c'est l'utilisateur qui place ses rappel selon un ordre arbitraire mais on pourrait pousser se système afin que l'utilisateur puisse ordonner ses rappels selon un ordre alphabétique, par thème, par date etc... En d'autre terme un système de tri et de filtrage.

Une autre piste d'amélioration concerne l'**interface graphique** : un design plus moderne et responsive, avec des icônes plus visuelles et une meilleure adaptation à différentes tailles d'écran, rendrait l'application encore plus agréable à utiliser.

Sur le plan technique, on pourrait envisager une **synchronisation des données** entre plusieurs appareils. Ici il s'agirait simplement d'implementer le shema de la base de donnée vu plus au dessus où la table des rappels serait relié a un login. Il suffirait d'ouvrir une fenetre au préalable avec des champs login et mot de passe a remplir.

6. Conclusion

6.1 Bilan de Camille

Ma conclusion est la suivante : Ce projet a été différent des précédents, notamment par les outils que nous avons utilisés. En effet, bien que l'objectif soit à nouveau de créer une application en Java, j'ai cette fois mobilisé plusieurs outils que j'ai découverts cette année, comme MariaDB ou encore d'autres outils de développement vus depuis le début de l'année.
Un outil, que je n'avais encore jamais utilisé dans mes projets précédents, a été particulièrement important : le Makefile.
Son utilisation s'est révélée très utile : même s'il a fallu un certain temps d'adaptation pour bien comprendre son fonctionnement, il nous a ensuite grandement facilité la tâche et nous a fait gagner du temps dans la compilation et l'exécution du projet.

6.2 Bilan de Benicia

Je réalise que mon approche d'un énoncé est encore à approfondir. Travailler avec mes camarades a été particulièrement bénéfique, car ils ont pu pallier certaines de mes lacunes et m'ont aidé à mieux comprendre des points qui n'étaient pas toujours clairs en cours. J'ai également remarqué que nous avions des approches différentes face au même énoncé et que la mienne était malheureusement insuffisante. Cependant, en échangeant avec eux, j'ai pu observer des méthodes plus efficaces et apprendre comment mieux appréhender un énoncé à l'avenir.

6.3 Bilan de Valentin

Ce projet m'a permis de consolider mes compétences en développement Java tout en découvrant de nouveaux outils et méthodes de travail. J'ai particulièrement apprécié la dimension collaborative du projet, qui m'a aidé à mieux comprendre certaines notions grâce aux échanges avec mes camarades.
La mise en place de la base de données MariaDB et l'utilisation du Makefile ont été des points nouveaux et formateurs pour moi. J'ai parfois rencontré des difficultés lors de la configuration et des tests, mais ces obstacles m'ont permis de mieux comprendre le fonctionnement global d'un projet logiciel structuré.
De manière générale, ce projet m'a appris à être plus rigoureux dans mon organisation et à adopter de bonnes pratiques de développement. C'était une expérience enrichissante, à la fois sur le plan technique et humain.